Import Functions

We begin by importing all the necessary function and libraries we will be using

In [1]:
```
import numpy as np
import os
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (GridSearchCV, StratifiedKFold, ParameterGri
from sklearn.metrics import (average_precision_score,
                             roc_auc_score,
                             recall_score,
                             accuracy_score,
                             precision_score,
                             f1_score,
                             matthews_corrcoef,
                             balanced_accuracy_score,
                             confusion_matrix, RocCurveDisplay, PrecisionRecallDi
```

Import Training Data

We import our training data which we have split from our dataset prior to this

In [2]:
```
train = pd.read_csv("training_data.csv")
TARGET = 'mortality'#for conveniance we call this target as it is what we are cl

X_train = train.drop(columns=[TARGET])#variables without the mortality we are pr
y_train = train[TARGET]#what we're predicting

print("Predictor columns:", X_train.shape[1])#check about the shape of our data
print("Missing target values:", y_train.isnull().sum())#are there any values we
```

```
Predictor columns: 107
Missing target values: 0
```

Imputation on Training Data

Since we have quite a few missing values we are going to have to impute our data.

Imputation is the method of substituting missing values for some other ones, based on the values that aren't missing. The most common method is using the median of existing values as this subsititution (for numerical data, for categorical we'd use the most frequent category- or the mode). This is what we will be using in this report

We learned from analysis of our data and the site where are data is from that some of our variables are categorical and some are numerical. We know which variables are numerical so we can label them as such and split our variables into numerical and categorical.

It is important to note that sometimes, missing data can be an important. It could indicate how the patient has been treated, it could be the result of wider contexts, therefore we may want to know if it was missing. Due the heavy computation we will do soon with our Random Forest Paramters, testing if missing data would be important for all of them would waste time and energy, once we have narrowed down our options we can then do so. Thus we intitally have no missingness indicators.

In [3]:
```
numerical_cols = ["AGE","S_AD_ORIT","D_AD_ORIT","K_BLOOD","NA_BLOOD","ALT_BLOOD"
categorical_cols = [col for col in X_train.columns #all other variables are cate
        if col not in numerical_cols]

print("Numerical variables:", len(numerical_cols))#can check our column amounts
print("Categorical variables:", len(categorical_cols))
```

```
Numerical variables: 9
Categorical variables: 98
```

We will be using K-Fold Cross Validation to tune our hyperparamters for the random forest classifer we will be using later on in this section. This interacts with imputation in a unique way, it means we need to impute in each fold, we can't impute first then use K-Fold CV as we'd be using data from other folds to affect each fold.

As a result we need to predefine the folds that we will be using later. I chose five splits as it will be more suitable with the amount of data we have, since our data has 16% mortality we'd have quite a small proportion of deaths in each fold if we had picked ten or more folds. Also due to this, we also use a stratified K-Fold which accounts for this heavy imbalance in our data.

In [4]:
```
fold_data = []#predfine for use in loop
cv = StratifiedKFold(n_splits=5,shuffle=True,random_state=1)

for train_idx, val_idx in cv.split(X_train, y_train):

    # Split our data into training folds and validation folds, we use copies of
    X_fold_train = X_train.iloc[train_idx].copy()#affect it
    X_fold_val = X_train.iloc[val_idx].copy()

    y_fold_train = y_train.iloc[train_idx]
    y_fold_val = y_train.iloc[val_idx]

    # Create new imputers for this fold, we use median and mode as explained bef
    num_imp = SimpleImputer(strategy="median")
    cat_imp = SimpleImputer(strategy="most_frequent")

    # Fit numerical imputer on fitting data only, here is where we keep the vali
    X_fold_train[numerical_cols] = num_imp.fit_transform(X_fold_train[numerical_
    # Apply the same numerical imputation to validation fold
    X_fold_val[numerical_cols] = num_imp.transform(X_fold_val[numerical_cols])

    # Repeat for categorical data
    X_fold_train[categorical_cols] = cat_imp.fit_transform(X_fold_train[categori
    X_fold_val[categorical_cols] = cat_imp.transform(X_fold_val[categorical_cols

    # Save this prepared fold
    fold_data.append((X_fold_train,X_fold_val,y_fold_train,y_fold_val))#remember
```

```
                        #values, so doesn't need to be imputed we just need to mai

print("Number of prepared folds:", len(fold_data))#check we have the right amoun
```

```
Number of prepared folds: 5
```

We can do a quick sanity check to make sure our imputation per fold has worked

In [5]:
```
for i, (X_tr, X_val,y_tr, y_val) in enumerate(fold_data):
    print(f"Fold {i+1}:",
        "Training missing =", X_tr.isna().sum().sum(),"| Validation missing =",
```

```
Fold 1: Training missing = 0 | Validation missing = 0
Fold 2: Training missing = 0 | Validation missing = 0
Fold 3: Training missing = 0 | Validation missing = 0
Fold 4: Training missing = 0 | Validation missing = 0
Fold 5: Training missing = 0 | Validation missing = 0
```

Random Forest Classifier

In this report I will be using a Random Forest Classifer. A Random Forest can have many different parameters and variables, and as part of my investigation I want to decide which ones to use. Reading the documentation on the random forest in SK-Learn, we learn that important ones to consider are the number of trees in the forest (n_estimators) and the number of features considered at each leaf node (max_features). I chose to stick to n_estimators=300 as it is a good balance between more information and saving computer power. We can tune for other paramters such as:

- max_features: maximum number of features considered for splitting a node.
- max_depth: maximum number of levels in each decision tree.
- min_samples_leaf: mininum number of data points allowed in a leaf node.
- class_weight: do we balance each subsample based on our data split, especially important here due to our aforementioned data imbalance.

The way we determine the best value for these is to test different models with all these different paramter combinations and see how they perform- then decide which is best. Below we can see which options we our testing for each paramter, and how many combinations there are.

In [6]:
```
param_grid = {'max_depth': [3, 5, 10, None],#here we describe the different opti
              'min_samples_leaf': [1, 5, 10],
              'max_features': ['sqrt','log2', 0.5, None],
              'class_weight': [None, 'balanced_subsample']}#since mortality is o


parameter_combinations = list(ParameterGrid(param_grid))
print("Number of combinations:", len(parameter_combinations))
```

```
Number of combinations: 96
```

Choosing how to choose 'best'

There are many different ways to test performance for our model. We have chosen a few below, each has positives and negatives in a range of contexts. We will use some of these and others at the end to compare the model from this section to other models.

The performance metrics we will be using are below:

- Accuracy: The proportion of all predictions that the model classifies correctly, bare in mind it can be misleading when the dataset is imbalanced like here.
- Precision: The proportion of predicted positive cases that are actually positive, indicating how reliable the model's positive predictions are. In our case this means how many deaths we predict actually occur.
- Recall: The proportion of actual positive cases that are correctly identified, particularly important here since a false negative results in a death we didn't predict.
- F1-score: The harmonic mean of precision and recall.
- ROC-AUC: The area under our ROC curve, it measures how effectively the model distinguishes between positive and negative classes across different classification thresholds.
- Average Precision (AP): Averages precision weighted by increases in recall, measuring how effectively the model identifies and ranks positive cases across different thresholds.

All of these are important, we could argue that Recall is more important in our medical context as we ideally want to minimise missed deaths, however if it comes at a huge cost to other metrics we might not want to choose it as our performance indicator. We will explore what hyperparameters we should pick if we were to choose different metrics.

Here is where our K-Fold CV is used to check performance, there is a function that would do this normally however it doesn't allow us to impute at each step, hence our need to define folds earlier and to do the search by hand below.

In [7]:
```
metrics = ['accuracy','precision','recall','f1','roc_auc','average_precision']
```

In [8]:
```
results = []#store for later
for i, params in enumerate(parameter_combinations):
    # Store scores for each fold
    fold_scores = {metric: [] for metric in metrics}
    for X_tr, X_val, y_tr, y_val in fold_data:#this is where we use our K-Fold V
        # Define Random Forest with current parameters
        rf = RandomForestClassifier(n_estimators=300,random_state=1,bootstrap=Tr
        # Fit on this fold's training data
        rf.fit(X_tr, y_tr)
        # Predict mortality probabilities
        probabilities = rf.predict_proba(X_val)[:, 1]
        # Convert probabilities to 0/1 predictions
        predictions = (probabilities >= 0.5).astype(int)
        # Calculate performance metrics
        fold_scores['accuracy'].append(accuracy_score(y_val, predictions))
        fold_scores['precision'].append(precision_score(y_val, predictions, zero
        fold_scores['recall'].append(recall_score(y_val, predictions))
        fold_scores['f1'].append(f1_score(y_val, predictions, zero_division=0))
        fold_scores['roc_auc'].append(roc_auc_score(y_val, probabilities))
        fold_scores['average_precision'].append(average_precision_score(y_val, p
```

```
    # Calculate mean and standard deviation across folds
    row = {'params': params}

    for metric in metrics:
        row[f'mean_{metric}'] = np.mean(fold_scores[metric])
        row[f'std_{metric}'] = np.std(
            fold_scores[metric], ddof=1)
    results.append(row)

    # Print progress every ten configurations
    if (i + 1) % 10 == 0:
        print(f"Completed {i+1} of {len(parameter_combinations)}")

# Convert all results into a dataframe
results_df = pd.DataFrame(results)

print("Search complete!")
```

```
Completed 10 of 96
Completed 20 of 96
Completed 30 of 96
Completed 40 of 96
Completed 50 of 96
Completed 60 of 96
Completed 70 of 96
Completed 80 of 96
Completed 90 of 96
Search complete!
```

Now that we have gone through all our combinations and tested for all our metrics, we can explore our results.

Ideal Random Forest Hyperparamters for each performance metric

In [9]:
```
best_models = {}

for metric in metrics:#go through our results for each metric
    best_index = results_df[f'mean_{metric}'].idxmax()
    best_row = results_df.loc[best_index]
    best_models[metric] = best_row['params']
    print(f"\nBest model for {metric}:")
    print(best_row['params'])
    print("Mean score:",round(best_row[f'mean_{metric}'], 4))
```

```
Best model for accuracy:
{'class_weight': 'balanced_subsample', 'max_depth': 10, 'max_features': 'log2',
'min_samples_leaf': 5}
Mean score: 0.898

Best model for precision:
{'class_weight': None, 'max_depth': 3, 'max_features': 'sqrt', 'min_samples_lea
f': 1}
Mean score: 0.9667

Best model for recall:
{'class_weight': 'balanced_subsample', 'max_depth': 3, 'max_features': 'log2', 'm
in_samples_leaf': 10}
Mean score: 0.6883

Best model for f1:
{'class_weight': 'balanced_subsample', 'max_depth': 10, 'max_features': 'log2',
'min_samples_leaf': 10}
Mean score: 0.6297

Best model for roc_auc:
{'class_weight': None, 'max_depth': 10, 'max_features': 'log2', 'min_samples_lea
f': 5}
Mean score: 0.8939

Best model for average_precision:
{'class_weight': 'balanced_subsample', 'max_depth': None, 'max_features': 'log2',
'min_samples_leaf': 5}
Mean score: 0.6977
```

Exploration of results

Accuracy focused: Our accuracy-optimised Random Forest correctly classifies approximately 89.8% of held-out patients. This is higher than the approximate 84% accuracy we could get by predicting survival for every patient (since that many survive), however it doesn't tell us if we are predicting death well- which is what we'd want to focus on)

Precision: This seems quite promising as we achieve an average precision score of approximately 96.67%. This means that when we predicts mortality, our positive predictions are almsot always correct. There may be a problem though, we could miss a lot of deaths, i.e. if we could predict that only two patients will die, and are correct we'd achieve 100% precision while missing other deaths.

Recall: Our recall focused Random Forest identifies approximately 69% of the patients who die, this does mean we miss 31% of deaths. We need to see how increasing recall might lead to more patients being incorrectly identified as high risk. Then decide what balance we want.

F1-score: This is the balance between precision and recall we meantioned above and we achieved 62.97%.

ROC-AUC: The area under our ROC curve was approximately 0.894, this suggests that the model has a strong ability to rank patients who die above those who survive. However,

unlike the above metrics this doesn't give us a specific probability threshold to use, the other require one and we chose the standard 0.5.

Average precision — 69.77% Our Average precision was 69.77%, with no single threshold like ROC-AUC this evaluates how well the model identifies and ranks mortality cases across thresholds. Which is quite reassuring as mortality is only 16% of our data.

As we can see, each of these are important in different ways, but we still need wider context to make a decison. We need to see how each 'best' model performs on the other metrics it didn't use to score itself on.

In [10]:
```
comparison = []

for metric in metrics:#here we don't look at the features we just check how each

    best_index = results_df[f'mean_{metric}'].idxmax()
    best_row = results_df.loc[best_index]

    comparison.append({
        'Selection metric': metric,
        'Best parameters': best_row['params'],
        'Accuracy': best_row['mean_accuracy'],
        'Precision': best_row['mean_precision'],
        'Recall': best_row['mean_recall'],
        'F1': best_row['mean_f1'],
        'ROC-AUC': best_row['mean_roc_auc'],
        'Average precision': best_row['mean_average_precision']})

comparison_df = pd.DataFrame(comparison)
display(comparison_df.round(3))
```

	Selection metric	Best parameters	Accuracy	Precision	Recall	F1	ROC-AUC	Aver preci
0	accuracy	{'class_weight': 'balanced_subsample', 'max_de...	0.898	0.738	0.554	0.627	0.891	0
1	precision	{'class_weight': None, 'max_depth': 3, 'max_fe...	0.861	0.967	0.135	0.234	0.875	0
2	recall	{'class_weight': 'balanced_subsample', 'max_de...	0.862	0.557	0.688	0.614	0.885	0
3	f1	{'class_weight': 'balanced_subsample', 'max_de...	0.877	0.612	0.658	0.630	0.889	0
4	roc_auc	{'class_weight': None, 'max_depth': 10, 'max_f...	0.865	0.938	0.166	0.276	0.894	0
5	average_precision	{'class_weight': 'balanced_subsample', 'max_de...	0.898	0.743	0.548	0.626	0.892	0

We can learn quite a few things from the table above. Many of our assumptions and worries were proven correct and we can use this to decide which metric (and hence hyperparameters) to use.

One of the things we can see immediately is that high precision doesn't necessarily mean a good mortality model. We achieved high precision in it's model but now we can see that the recall is 0.135. This means we miss around 86.5% of deaths. Since our intention is to identify patients at risk, this could make the model unsuitable, even though with such high precision.

This does clearly show why we've explored different performance metrics- as using one can be misleading.

We've discussed how important recall is in our context but we can see the recall optimised model has a precision of 55.7%. Whilst great at predicting people at risk of death, almost half of our patients predicted to die end up surviving. In the real world we know this could come at cost of unnecessary investigations and clinical resources- which could waste money and time.

One last important thing we can explicitly point out here is in our ROC-AUC model. It achieves a ROC-AUC of 0.894, Precision of 0.938, and Recall of 0.166. The area under the curve of 0.894 indicates the model generally assigns higher mortality risk scores to patients who die than to patients who survive. However the recall is only 0.166. This leads to a question regarding the default threshold of 0.5, it might be it needs changing (we end up looking at this later on).

In conclusion: no single performance metric tells the complete story, we need to pick a metric based on the context of our data and what we ultimately want it to achieve, we have reason the think that our classification threshold may be more important than we assumed and may need to explore changing it.

Short-listing models for validation

With all the information we have discussed before we are ready to short list some models' hyperparamter combinations. Then we can finally decide based on some metric in the validation stage.

Due to the considerable overlap, one way we could choose is simply picking the hyperparamter choice the showed up the most in across each best model. When doing so we'd get:

- class_weight: 'balanced_subsample',
- max_depth: 10,
- max_features: 'log2',
- min_samples_leaf: 5

We can see that this is just the accuracy model. This choice is intuitive, however, such an approach ignores interactions between hyperparameters and treats each metric as equally informative. For this reason we choose two other models to take with us to the validation stage:

- The F1-optimised forest, it's reasonable compromise, with recall and precision of 0.658 and 0.612 respecitvely.
- Average-precision-optimised forest, it had (unsurprisingly) the best average precision of 0.698 and joint-highest reported accuracy (0.898)-the downside being it's recall of 0.548

Going forward we will use the average precision as our choice of comparing RF models. Alongside the benefits it has that I mentioned earlier, it will allow us to more cleanly address how ultimately changing the threshold for our model could impact our other metrics.

In [11]:
```
candidate_models = {
    "AP":{"max_depth": None, "min_samples_leaf": 5, "max_features": "log2", "cla
    "F1":{"max_depth": 10, "min_samples_leaf": 10, "max_features": "log2", "clas
    "Accuracy":{"max_depth": 10, "min_samples_leaf": 5, "max_features": "log2",
```

Testing for the inclusion of Missingness indicators

We mentioned before how missing measurements (especially from hospitals) might contain additional predictive information. Now that we have narrowed down our hyperparamter choices we can actually check for this.

We introduce missingness indicators for variables with at least 10% missing observations. Then use our three shortlisted Random Forest configurations to evaluate changes in average precision when their hyperparameters are fixed.

We test by use 5-Fold cross validation like before for our hyperparameters.

In [44]:
```
results=[]

for fold, (train_idx, val_idx) in enumerate(#for each fold like before
    cv.split(X_train, y_train), start=1):

    #We first split our data into their training and validation folds
    X_tr = X_train.iloc[train_idx].copy()
    X_val = X_train.iloc[val_idx].copy()
    y_tr = y_train.iloc[train_idx]
    y_val = y_train.iloc[val_idx]

    #Select columns needing missingness indicators, we only use the training fol
    missing_fraction = X_tr.isna().mean()
    flag_cols = missing_fraction[missing_fraction >= 0.10].index.tolist()#add in

    #We create imputers for this fold
    num_imp = SimpleImputer(strategy="median")
    cat_imp = SimpleImputer(strategy="most_frequent")
    X_tr_imp = X_tr.copy()#make copies so we don't change original data
    X_val_imp = X_val.copy()

    #Fit imputation like before
    X_tr_imp[numerical_cols] = num_imp.fit_transform(X_tr[numerical_cols])
    X_val_imp[numerical_cols] = num_imp.transform(X_val[numerical_cols])

    X_tr_imp[categorical_cols] = cat_imp.fit_transform(X_tr[categorical_cols])
    X_val_imp[categorical_cols] = cat_imp.transform(X_val[categorical_cols])

    #We create copies for indicator-based models
    X_tr_flags = X_tr_imp.copy()
    X_val_flags = X_val_imp.copy()

    #Add missing indicators based on our original unimputed missing values
    for col in flag_cols:
        X_tr_flags[col + "_missing"] = (X_tr[col].isna().astype(int))
        X_val_flags[col + "_missing"] = (X_val[col].isna().astype(int))

    #Now we test each candidate model
    for model_name, params in candidate_models.items():
        for use_flags in [False, True]:
            if use_flags:
                X_fit = X_tr_flags
                X_evaluate = X_val_flags
            else:
                X_fit = X_tr_imp
                X_evaluate = X_val_imp

            rf = RandomForestClassifier(#define our random forest
                n_estimators=300,
                bootstrap=True,
                criterion="gini",
                random_state=1,
```

```
                n_jobs=1,
                **params)
            rf.fit(X_fit, y_tr)#fit model and evaluate it's performance using AP
            probabilities = rf.predict_proba(X_evaluate)[:, 1]
            ap = average_precision_score(y_val, probabilities)
            results.append({"Model": model_name, "Fold": fold, "Indicators": use

    print(f"Fold {fold} completed")#progress check

results_df = pd.DataFrame(results)#store our results
```

```
Fold 1 completed
Fold 2 completed
Fold 3 completed
Fold 4 completed
Fold 5 completed
```

We can view the impact of adding missingness indicators by displaying our results for each model and comparing the means and standard deviations of our folds with or without missingness indicators (shown by True and False)

In [45]:
```
summary=results_df.groupby(["Model", "Indicators"])["Average_Precision"].agg(["m
display(summary.round(4))#display results in table
```

		mean	std
Model	**Indicators**		
AP	**False**	0.6977	0.0825
	True	0.7060	0.0910
Accuracy	**False**	0.6958	0.0932
	True	0.7033	0.0970
F1	**False**	0.6811	0.0960
	True	0.6875	0.0996

The differences appear to be quite small but since these are based on all the folds maybe in some cases (in particular folds) there is a great impact we we have lost it in the summary. Hence let's look at how each fold's mean is affected, we can also include the actual difference.

In [46]:
```
comparison=results_df.pivot(index=["Model", "Fold"],columns="Indicators",values=
comparison=comparison.rename(columns={False: "Without indicators",True: "With in
comparison["Difference"]=(comparison["With indicators"] - comparison["Without in

display(comparison.round(4))
```

Model	Fold	Indicators	Without indicators	With indicators	Difference
AP	**1**		0.6647	0.6334	-0.0313
	2		0.6448	0.6637	0.0189
	3		0.6137	0.6245	0.0109
	4		0.8125	0.8134	0.0008
	5		0.7531	0.7950	0.0419
Accuracy	**1**		0.6471	0.6197	-0.0274
	2		0.6343	0.6713	0.0370
	3		0.6088	0.6125	0.0037
	4		0.8223	0.8023	-0.0201
	5		0.7665	0.8108	0.0442
F1	**1**		0.6313	0.6083	-0.0229
	2		0.6098	0.6594	0.0496
	3		0.5978	0.5851	-0.0127
	4		0.8104	0.8005	-0.0100
	5		0.7562	0.7844	0.0282

Clearly no particualr folds stands out too much from the rest, so we can just look at absolute difference between each value per model.

In [47]:
```
mean_difference = comparison.groupby(level="Model")["Difference"].agg(["mean", "
display(mean_difference.round(4))
```

Model	mean	std
AP	0.0082	0.0268
Accuracy	0.0075	0.0325
F1	0.0064	0.0309

The inclusion of missingness indicators resulted in small changes in mean average precision: +0.0082 for the AP-optimised model, +0.0075 for the accuracy-optimised model, and −0.0029 for the F1-optimised model. These differences were relatively small compared with the variability observed between folds.

The suggests that missingness indicators provide little predictive benefit. From this we can disregard including additional indicators and keep the simpler median/mode imputation approach.

Now that we have addressed the missingness indicators, and have our shortlisted combinations for hyperparameters, we can move on to validation where we will pick which hyperparamters to ultimately use,

Validation

Import validation data

In [48]:
```
validate = pd.read_csv("validation_data.csv")

X_val = validate.drop(columns=[TARGET])#variables without the mortality we are p
y_val = validate[TARGET]#what we're predicting
```

Impute validation data

We can now impute for our validation data, for conveniance we make a 'final' training imputed set and final validation imputed set.

In [49]:
```
num_imp = SimpleImputer(strategy="median")#define imputers
cat_imp = SimpleImputer(strategy="most_frequent")

X_train_final = X_train.copy()#make copies
X_val_final = X_val.copy()

#Impute categorical and numerical variables
X_train_final[numerical_cols] = num_imp.fit_transform(X_train[numerical_cols])
X_val_final[numerical_cols] = num_imp.transform(X_val[numerical_cols])

X_train_final[categorical_cols] = cat_imp.fit_transform(X_train[categorical_cols
X_val_final[categorical_cols] = cat_imp.transform(X_val[categorical_cols])

print("Training missing:", X_train_final.isna().sum().sum())#sanity check
print("Validation missing:", X_val_final.isna().sum().sum())
```

```
Training missing: 0
Validation missing: 0
```

Validation testing

Evaluating how hyperparamter choices perform on validation data

We can finally train our models and see how they do on our validation data, we can check their performance on all the metrics we used earlier to make a more balanced decision on which hyperparamters to pick.

In [50]:
```
validation_results = []
for name, params in candidate_models.items():
    rf = RandomForestClassifier(
        n_estimators=300,
        bootstrap=True,
```

```
        criterion="gini",
        random_state=1,
        n_jobs=1,
        **params)
    # Train on final training set
    rf.fit(X_train_final, y_train)

    # Predict on unused final validation set
    probs = rf.predict_proba(X_val_final)[:, 1]
    preds = rf.predict(X_val_final)
    validation_results.append({#see how models perform
        "Model": name,
        "AP": average_precision_score(y_val, probs),
        "ROC-AUC": roc_auc_score(y_val, probs),
        "Recall": recall_score(y_val, preds),
        "Precision": precision_score(y_val, preds, zero_division=0),
        "F1": f1_score(y_val, preds),
        "Accuracy": accuracy_score(y_val, preds)})

validation_results_df = pd.DataFrame(validation_results)
display(validation_results_df.sort_values("AP", ascending=False).round(4))
```

	Model	AP	ROC-AUC	Recall	Precision	F1	Accuracy
0	AP	0.6311	0.8691	0.4444	0.6316	0.5217	0.8706
2	Accuracy	0.6259	0.8693	0.4444	0.6000	0.5106	0.8647
1	F1	0.6066	0.8575	0.5741	0.5439	0.5586	0.8559

From these results we can choose our model. In the end I chose the AP focused model, it gives us a lower recall than F1, but we gain a lot more in precision by comparison, which is a worthwile tradeoff for now.

In [51]:
```
best_model_name = validation_results_df.sort_values("AP", ascending=False).iloc[
best_params = candidate_models[best_model_name]
print("Selected model:", best_model_name)
print("Selected parameters:", best_params)
```

```
Selected model: AP
Selected parameters: {'max_depth': None, 'min_samples_leaf': 5, 'max_features':
'log2', 'class_weight': 'balanced_subsample'}
```

The benefit of our AP selected model is it doesn't have fixed threshhold. The model has a validation ROC-AUC of 0.8691, suggesting reasonably strong discrimination, but its recall at the default 0.5 threshold is only 0.4444. This leads us on to how we can change the threshold to see see if that could make our model better. First we train our chosen best random forest on the training data.

In [52]:
```
best_rf = RandomForestClassifier(
    n_estimators=300,
    bootstrap=True,
    criterion='gini',
    random_state=1,
    n_jobs=1,
    **best_params)
```

```
# Fit on training data
best_rf.fit(X_train_final, y_train)

# Predict probabilities on validation data
val_probabilities = best_rf.predict_proba(X_val_final)[:, 1]
```

Evalutating model on different thresholds

We use a whole range of thresholds from 0.2 to 0.75, using steps of 0.05. For each threshold we check our performance anew and display the results in a table.

In [53]:
```
thresholds = np.arange(0.2, 0.76, 0.05)#array of differnt thresholds
threshold_results = []
for threshold in thresholds:
    # Classify using the chosen threshold
    predictions = (val_probabilities >= threshold).astype(int)
    # Calculate confusion matrix values
    tn, fp, fn, tp = confusion_matrix(y_val, predictions, labels=[0, 1]).ravel()

    # Calculate performance metrics
    threshold_results.append({
        'Threshold': threshold,
        'Accuracy': accuracy_score(y_val, predictions),
        'Precision': precision_score(y_val, predictions, zero_division=0),
        'Recall': recall_score(y_val, predictions),
        'F1': f1_score(y_val, predictions),
        'False positives': fp,
        'False negatives': fn})

threshold_df = pd.DataFrame(threshold_results)

display(threshold_df.round(4))
```

	Threshold	Accuracy	Precision	Recall	F1	False positives	False negatives
0	0.20	0.4088	0.2118	1.0000	0.3495	201	0
1	0.25	0.5353	0.2500	0.9630	0.3969	156	2
2	0.30	0.6588	0.3062	0.9074	0.4579	111	5
3	0.35	0.7471	0.3710	0.8519	0.5169	78	8
4	0.40	0.8059	0.4318	0.7037	0.5352	50	16
5	0.45	0.8441	0.5077	0.6111	0.5546	32	21
6	0.50	0.8706	0.6316	0.4444	0.5217	14	30
7	0.55	0.8853	0.8571	0.3333	0.4800	3	36
8	0.60	0.8588	0.8750	0.1296	0.2258	1	47
9	0.65	0.8618	1.0000	0.1296	0.2295	0	47
10	0.70	0.8618	1.0000	0.1296	0.2295	0	47
11	0.75	0.8559	1.0000	0.0926	0.1695	0	49

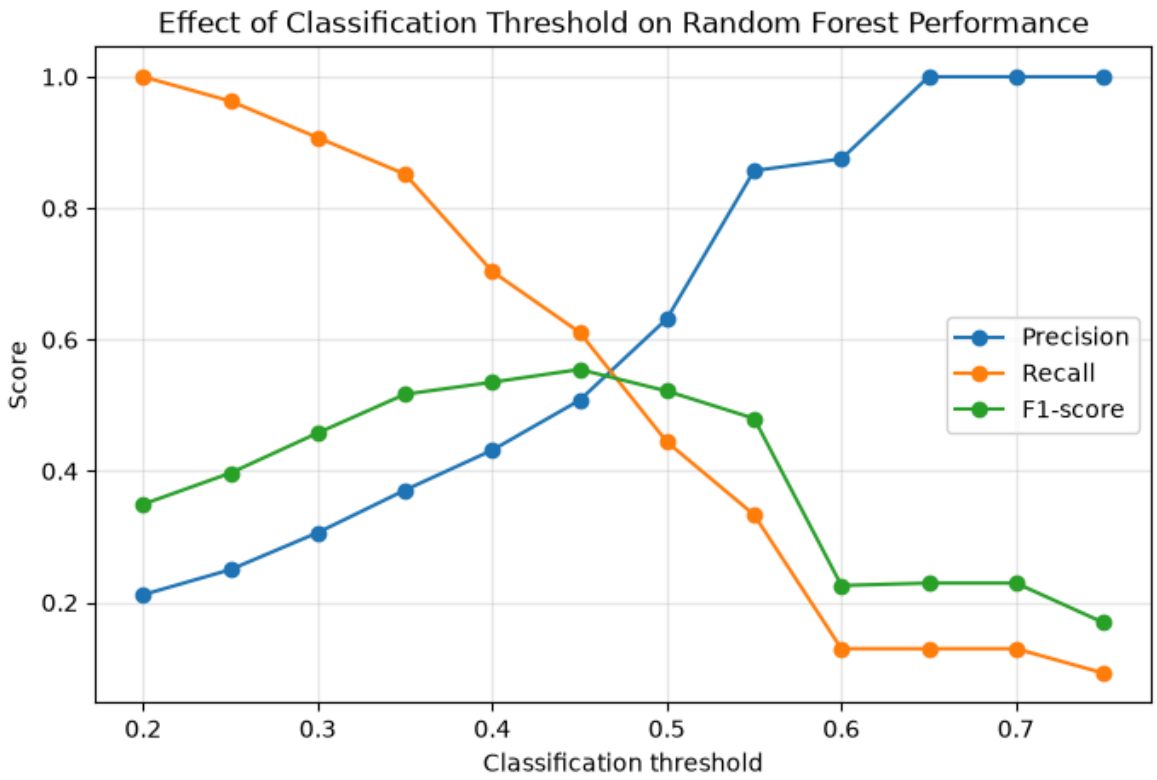
We can pay particular focus to the False positive and False negative counts, as mentioned before we want False negatives to be low in mortality data- something that would inherently bias recall. But too low gives us incredibly high false positives. 0.35-0.45 seems to be our range of interest (before that we have over 100 False positives which seems large with a very low precision of around 0.3, and after this we have recall lower than 0.5 so we miss more than half deaths and in fact our False negatives would be higher than our False positives- neither of which seems suitable)

To help visualise this table we can plot how Precision, Recall and F1 all behave against Threshold change:

In [54]:
```
plt.figure(figsize=(8, 5))
plt.plot(threshold_df['Threshold'], threshold_df['Precision'], marker='o', label
plt.plot(threshold_df['Threshold'], threshold_df['Recall'], marker='o', label='R
plt.plot(threshold_df['Threshold'], threshold_df['F1'], marker='o', label='F1-sc

plt.xlabel('Classification threshold')
plt.ylabel('Score')
plt.title('Effect of Classification Threshold on Random Forest Performance')

plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Choosing classification threshold sensitivity

In the end I made the choice of choosing a threshold of 0.35.

Lowering the threshold from 0.45 to 0.35 increases recall from 0.648 to 0.852, allowing the Random Forest to correctly identify more of the mortality cases. This improvement

comes at the expense of precision, which decreased from 0.530 to 0.374, increasing False positive predictions.

We demonstrate that the most appropriate threshold depends on the consequences of different classification errors. In our case of mortality predicition, identifying patients at risk is what we prioritised over avoiding False positive predictions.

With our hyperparameters all decided we are ready to evaluate our model on our unseen test data.

Final Random Forest Test Evaluation

Import test data

In [55]:
```
test = pd.read_csv("test_data.csv")
```

In [56]:
```
X_test = test.drop(columns=["mortality"]).copy()#split test data into target and
y_test = test["mortality"].copy()

# Check the test data contains the required features
missing_cols = X_train.columns.difference(X_test.columns)
print("Missing columns:", missing_cols.tolist())
if len(missing_cols) > 0:
    raise ValueError("Test dataset is missing required predictors")

X_test = X_test[X_train.columns].copy()#define test data, the columns we used in
print("Test predictor shape:", X_test.shape)#sanity check for sizes and shapes
print("Training predictor shape:", X_train.shape)
assert y_test.notna().all()
assert set(y_test.unique()).issubset({0, 1})
```

```
Missing columns: []
Test predictor shape: (340, 107)
Training predictor shape: (1020, 107)
```

We again need to impute our missing test data. We also make final training data to use in this section

In [57]:
```
# Create imputers
num_imp_final = SimpleImputer(strategy="median")
cat_imp_final = SimpleImputer(strategy="most_frequent")
# Make copies
X_train_final = X_train.copy()
X_test_final = X_test.copy()

# The actual imputation
X_train_final[numerical_cols] = num_imp_final.fit_transform(X_train[numerical_co
X_test_final[numerical_cols] = num_imp_final.transform(X_test[numerical_cols])

X_train_final[categorical_cols] = cat_imp_final.fit_transform(X_train[categorica
X_test_final[categorical_cols] = cat_imp_final.transform(X_test[categorical_cols

print("Training missing values:",
      X_train_final.isnull().sum().sum())
```

```
print("Test missing values:",
      X_test_final.isnull().sum().sum())
```

```
Training missing values: 0
Test missing values: 0
```

Fit Final Random Forest Model on complete training data

In [58]:
```
final_rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_leaf=5,
    max_features="log2",
    class_weight="balanced_subsample",
    bootstrap=True,
    criterion="gini",
    random_state=1,
    n_jobs=1)

final_rf.fit(X_train_final, y_train)
```

Out[58]:

▾ RandomForestClassifier

▸ Parameters

▸ Fitted attributes

In [59]:
```
# Predicted probability of mortality
test_probabilities = final_rf.predict_proba(X_test_final)[:, 1]

# Convert probabilities to binary predictions using our new threshold of 0.35
threshold=0.35
test_predictions=(test_probabilities >= threshold).astype(int)
```

Analysis of performance

For comparative purpose to this section we will first use the same performance metrics we used earlier, then we can narrow down to ones we will use to compare different models across this report (some are the same, some are new).

In [60]:
```
test_results = {
    "Accuracy": accuracy_score(y_test, test_predictions),
    "Precision": precision_score(y_test, test_predictions, zero_division=0),
    "Recall": recall_score(y_test, test_predictions),
    "F1": f1_score(y_test, test_predictions),
    "ROC-AUC": roc_auc_score(y_test, test_probabilities),
    "Average Precision": average_precision_score(y_test, test_probabilities)}

test_results_df = pd.DataFrame(test_results.items(),columns=["Metric", "Test Sco
display(test_results_df.round(4))
```

	Metric	Test Score
0	Accuracy	0.7235
1	Precision	0.3413
2	Recall	0.7963
3	F1	0.4778
4	ROC-AUC	0.8559
5	Average Precision	0.6467

As expected we sacrificed a lot of our Precision, but we have a very high Recall, Accuracy and ROC-AUC. The F1 suffered due to it;s relationship with precision but now we can evaluate on metrics we will use in the conclusion:

In [61]:
```
final_test_results = {
    "Accuracy": accuracy_score(y_test, test_predictions),
    "Precision": precision_score(y_test, test_predictions, zero_division=0),
    "Recall": recall_score(y_test, test_predictions),
    "F1": f1_score(y_test, test_predictions),
    "ROC-AUC": roc_auc_score(y_test, test_probabilities),
    "mcc": matthews_corrcoef(y_test, test_predictions),
    "balanced_acc": balanced_accuracy_score(y_test, test_predictions)}

final_test_results_df = pd.DataFrame(final_test_results.items(),columns=["Metric
display(final_test_results_df.round(4))
```

	Metric	Test Score
0	Accuracy	0.7235
1	Precision	0.3413
2	Recall	0.7963
3	F1	0.4778
4	ROC-AUC	0.8559
5	mcc	0.3830
6	balanced_acc	0.7530

The extra metrics we have implemented are:

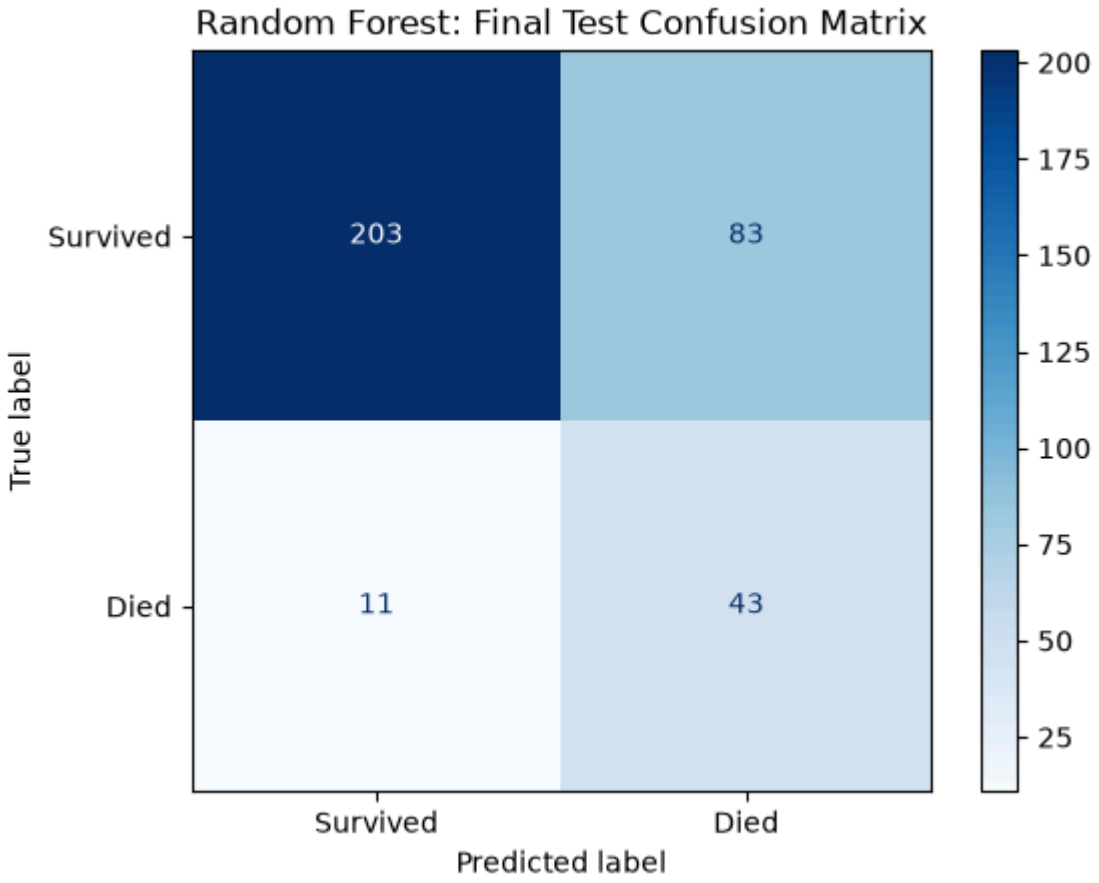
- Balanced Accuracy: Average of sensitivity (recall) and specificity.
- Matthews Correlation Coefficient: Overall quality of binary predictions, considering all four confusion-matrix outcomes.

The reason both of these are particularly useful here is that they account for our data being imbalanced as much as it is, making them more useful in comparing our models on this data.

Plots

In this last section we can construct some useful plots to help visualise our model performance. First we do a confusion matrix, then an ROC curve and finally a precision recall curve.

In [62]:
```
cm=confusion_matrix(y_test,test_predictions, labels=[0, 1])#get our confusion ma
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Survived", "Di
disp.plot(cmap="Blues")
plt.title("Random Forest: Final Test Confusion Matrix")
plt.show()
```

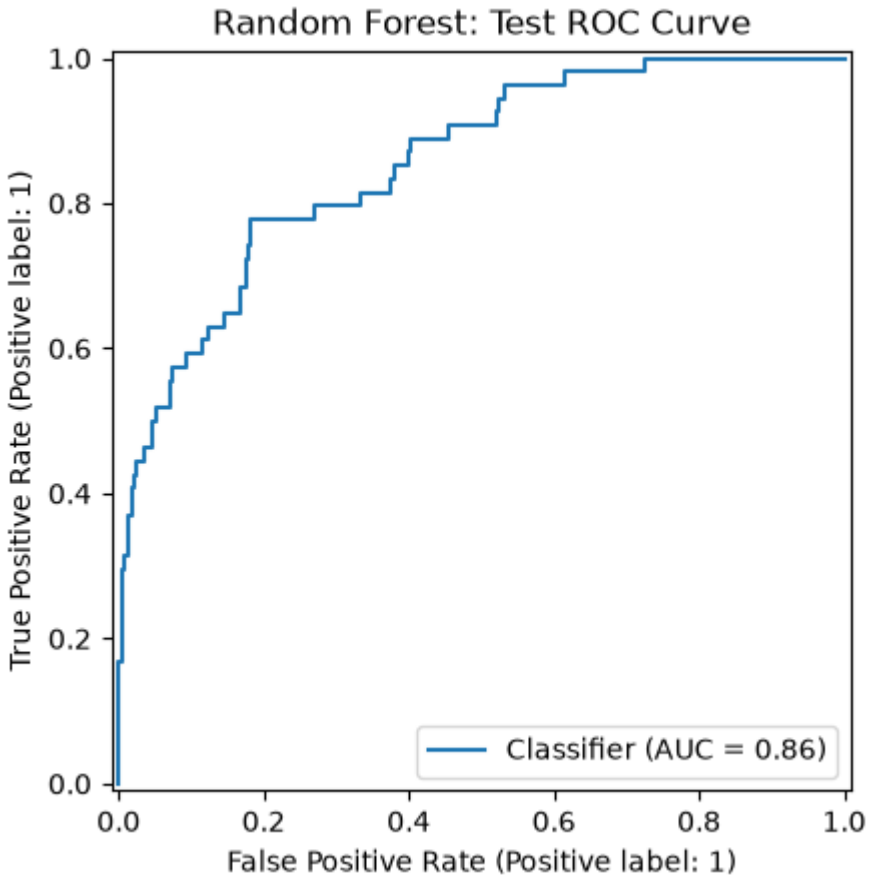


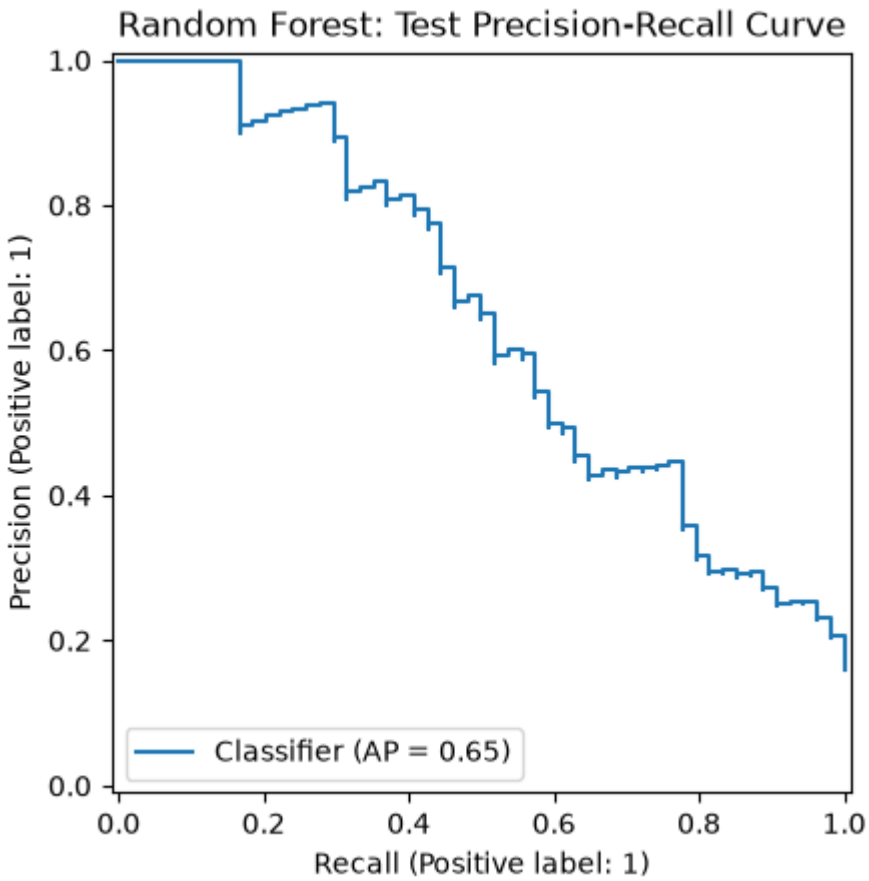
We can see that we very strongly predict survival on patients that survived, we also have an excellent performance for false negatives. Only 11 people died when we said they'd survive, though it is clear this caused our False Positives to spike as well as we have 83 patients who survived and we predicted they'd die.

Below you can see our ROC and Precision-Recall curves.

In [63]:
```
RocCurveDisplay.from_predictions(y_test, test_probabilities)
plt.title("Random Forest: Test ROC Curve")
plt.show()

PrecisionRecallDisplay.from_predictions(y_test, test_probabilities)
plt.title("Random Forest: Test Precision-Recall Curve")
plt.show()
```


Random Forest: Test ROC Curve
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
Classifier (AUC = 0.86)
0.0
0.2
0.4
0.6
0.8
1.0


Random Forest: Test Precision-Recall Curve
Precision (Positive label: 1)
Recall (Positive label: 1)
Classifier (AP = 0.65)
0.0
0.2
0.4
0.6
0.8
1.0

In general we can see that the model doesn't seem too bad based on the ROC curve, but the precision recall curve doesn't necesssarily indicate a better model. However these curves on their own don't provide us with much information, and are more useful when comparing models- which is what we'll do in our conclusion.

Concluding comments

To conclude this part of the wider report we can summarise what we've done here:

We explored how different performance metrics select different hyperparameters for our Random Forest Classifer, and showcased how for example optimising precision produces particularly poor recall. We ended up selecting the AP-optimised Random Forest, retained median/mode imputation without missingness indicators, and chose a classification threshold of 0.35 to prioritise mortality detection- given the context of our data.

References

- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- https://towardsdatascience.com/hyperparameter-tuning-the-random-forest-in-python-using-scikit-learn-28d2aa77dd74/
- https://medium.com/@priyankaparashar54/random-forest-classification-and-its-mathematical-implementation-f5e7e6878ca
- https://archive.ics.uci.edu/dataset/579/myocardial+infarction+complications